LOGIC BUILDING X RECURSION
(BEFORE STARTING DSA)

Phase 1 – Recursion Mindset Reset (No Math, No Strings)
Phase 2 – Liner Number Recursion (One Variable)
Phase 3 – Mathematical Thinking With Recursion
Phase 4 – Controlled Printing & Patterns
Phase 5 – String Recursion (Index Based Thinking)
Phase 6 – Array recursion (No Loops Allowed)

🎯***Goal:*** Master basic, linear recursion, index based thinking, array/string usecases
Topics covered: Recursion (Scratch to DSA Ready)
Target Questions: 100

Phase 1 – Recursion Mindset Reset (No Math, No Strings)

🎯***Goal:*** Understanding the core.

1. Write a recursive function that prints "Hello" exactly 5 times.
2. Print numbers from 1 to 5 using recursion **without parameters**.
3. Print numbers from 5 to 1 using recursion **without loops**.
4. Print "Before Call" and "After Call" to observe call order.
5. Print numbers from 1 to n **after the recursive call**.
6. Print numbers from 1 to n **before the recursive call**.
7. Print "Start" only once and "End" only once using recursion.
8. Print the current value of n during each recursive call.
9. Write a recursive function that stops only when n == 0.
10. Dry-run a recursive function manually and write call stack states.

Phase 2 – Liner Number Recursion (One Variable)

🎯***Goal:*** *Base + Recursive case mastery*

1. Print numbers from **1 to n** using recursion.
2. Print numbers from **n to 1** using recursion.
3. Print **even numbers** from 1 to n using recursion.
4. Print **odd numbers** from 1 to n using recursion.
5. Find the **sum of first n natural numbers** using recursion.
6. Find the **factorial of a number** using recursion.
7. Calculate **x^n (power of a number)** using recursion.
8. Find the **nth Fibonacci number** using recursion.
9. Print **Fibonacci series up to n terms** using recursion.

CodeWithNishchal

10. **Count digits** in a number using recursion.
11. Find the **sum of digits** of a number using recursion.
12. Find the **product of digits** of a number using recursion.
13. **Reverse a number** using recursion.
14. Check if a number is a **palindrome** using recursion.
15. Convert **decimal to binary** using recursion.
16. Convert **decimal to octal** using recursion.
17. Print **digits one by one** using recursion.
18. Print **digits in reverse order** using recursion.
19. Print **digits in words** using recursion.
20. **Count zeros** in a number using recursion.

Phase 3 – Mathematical Thinking With Recursion

🎯***Goal:*** *Breaking problems into sub-problems*

1. Find the GCD of two numbers using Euclid's algorithm.
2. Find the LCM of two numbers using recursion.
3. Find nCr using Pascal's relation recursively.
4. Find the sum of the first n even numbers using recursion.
5. Find the sum of the first n odd numbers using recursion.
6. Check whether a number is prime using recursion.
7. Count the number of prime digits in a given number.
8. Find the maximum and minimum digit in a given number.
9. Solve the Tower of Hanoi problem using a recursive approach.
10. Count the occurrences of a given digit in a number.
11. Check whether a number is an Armstrong number.
12. Find power using fast exponentiation recursively.
13. Calculate $\log_2(n)$ using recursion.
14. Count trailing zeros in the factorial of a number.
15. Find the digital root of a number using recursion.

Phase 4 – Controlled Printing & Patterns

🎯***Goal:*** *Recursive flow control without loops*

1. Print a line of n stars using recursion.
2. Print n lines of stars forming an n × n square pattern.
3. Print a right-angled triangle in top-down order using recursion.
4. Print a right-angled triangle in bottom-up order using recursion.
5. Print an inverted triangle pattern using recursion.
6. Print a pyramid pattern recursively.
7. Print the same number repeatedly on each row using recursion.
8. Print the multiplication table of a given number n.

CodeWithNishchal

9. Print multiplication tables from 1 to n using recursion.
10. Print numbers in increasing & then decreasing order using a single recursive function.
11. Print the sum series step-by-step using recursion.
12. Print patterns using two separate recursive functions.
13. Print patterns using only one recursive function.
14. Print a diagonal pattern using recursion.
15. Print a pattern using recursion with parameters.

Phase 5 – String Recursion (Index Based Thinking)

🎯***Goal:*** *Characters, indexes, sub-strings*

1. Print all characters of a given string using recursion.
2. Print all characters of a string in reverse order using recursion.
3. Reverse a given string using recursion.
4. Check whether a given string is a palindrome using recursion.
5. Count the number of vowels in a string using recursion.
6. Count the number of consonants in a string using recursion.
7. Remove all spaces from a given string using recursion.
8. Remove all vowels from a string using recursion.
9. Replace all occurrences of a given character in a string.
10. Remove all occurrences of a given character from a string.
11. Count the frequency of a given character in a string.
12. Convert all characters of a string to uppercase using recursion.
13. Convert all characters of a string to lowercase using recursion.
14. Check whether a string contains only digits using recursion.
15. Check whether a string contains only alphabets using recursion.
16. Count the number of words in a string using recursion.
17. Remove duplicate characters from a string using recursion.
18. Print a string without using the length() function.
19. Compare two strings recursively.
20. Check whether two strings are anagrams using recursion.

Phase 6 – Array Recursion (No Loops Allowed)

🎯***Goal:*** *Array traversal using index recursion*

1. Print all elements of an array using recursion.
2. Print all elements of an array in reverse order using recursion.
3. Find the sum of all elements in an array using recursion.
4. Find the maximum element present in an array using recursion.
5. Find the minimum element present in an array using recursion.

CodeWithNishchal

6. Count the number of even elements in an array using recursion.
7. Count the number of odd elements in an array using recursion.
8. Check whether the given array is sorted in ascending order using recursion.
9. Find the first occurrence of a given element in an array using recursion.
10. Find the last occurrence of a given element in an array using recursion.
11. Count the total occurrences of a given element in an array using recursion.
12. Reverse the given array using recursion.
13. Check whether the given array is a palindrome using recursion.
14. Replace all even numbers in the array with 0 using recursion.
15. Find the product of all elements of an array using recursion.
16. Print alternate elements of an array using recursion.
17. Find the index of the maximum element in the array using recursion.
18. Find the index of the minimum element in the array using recursion.
19. Check whether the array contains duplicate elements using recursion.
20. Merge two arrays into a single array using recursion.

Keep Coding, Grind Hard and **Connect With Me** on Social Media.

SHOW SOME LOVE BY FOLLOWING **CodeWithNishchal:**

- **Instagram:** https://www.instagram.com/codewithnishchal/
- **Book a 1:1 on Topmate:** https://topmate.io/nishchal_muradia
- **LinkedIn:** https://www.linkedin.com/in/nishchal-muradia/
- **YouTube Videos :** https://www.youtube.com/@CodeWithNishchal
- **Watch My Latest Time Complexity Series: Time Complexity**
- Telegram: https://t.me/codewithnishchal

CodeWithNishchal